SLURM
SLURM
DAGSTER

Why orchestration matters

- A raw “engine” (script, notebook, or binary) is not enough once you depend on sensors, ETL, or ML training.
- Orchestrators provide dependency tracking, retries, metrics, and the control plane HPC teams lack.
- Dagster supplies that missing layer; dagster-slurm connects it to the supercomputer’s scheduler.

Asset code stays the same

- Same Dagster asset runs locally or on Slurm; only configuration changes.
- Compute resource wraps payload execution with observability and retries.

```python
import dagster as dg
from dagster_slurm import BashLauncher, ComputeResource

@dg.asset
def train_pytorch(context: dg.AssetExecutionContext, compute: ComputeResource):
    script_path = dg.file_relative_path(
        __file__, "../workloads/pytorch/train_classifier.py"
    )

    completed = compute.run(
        context=context,
        payload_path=script_path,
        launcher=BashLauncher(),
        extra_env={"EPOCHS": "3", "BATCH_SIZE": "128"},
    )
    yield from completed.get_results()
```

Asset code stays the same

- Same Dagster asset runs locally or on Slurm; only configuration changes.
- Compute resource wraps payload execution with observability and retries.

```python
def main():
    context = PipesContext.get()
    num_epochs = context.extras["epochs"]
    context.log.debug(f"Number of epochs: {num_epochs}")
    input_path = os.environ.get("INPUT_DATA")
    context.log.info(f"Input: {input_path}")
    # Your processing logic here (pytorch, ... )
    result = {"rows_processed": 1000}
    context.report_asset_materialization(
        metadata={
            "rows": result["rows_processed"],
            "processing_time": "10s",
        }
    )

if __name__ == "__main__":
    with open_dagster_pipes() as context:
        main()
```

Different strengths, better together

Data orchestrator

- Models business data products and when they need to refresh.
- Captures lineage, metadata, and failures across heterogeneous systems.
- Keeps engineers productive with local runs, tests, and incremental deploys.

Supercomputer + Slurm

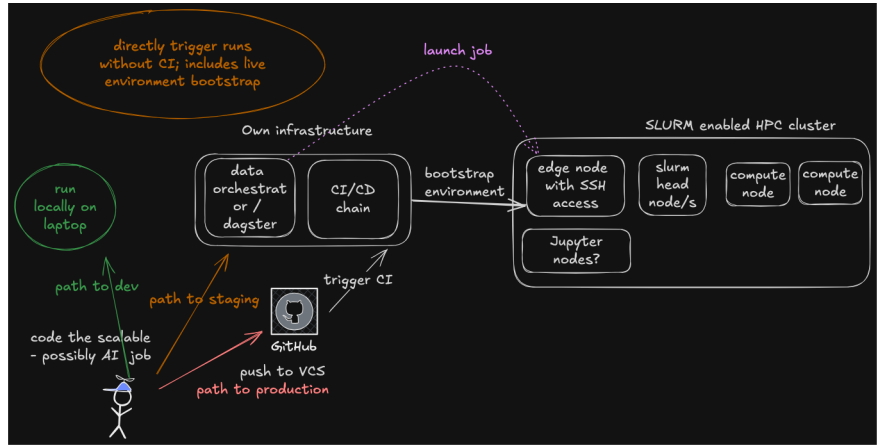
- Maximises utilisation of scarce accelerators and node hours.
- Enforces fair-share policies, job placement, and low-level resource binding.
- Provides high-performance file systems and interconnect-aware scheduling.

Bridge

- dagster-slurm lets Dagster plan the work while Slurm owns the physical execution.
- One orchestrator spans tools silos (HPC and non-HPC) for unified visibility

dagster-slurm in one slide

- **Keep the Dagster control plane** – asset logic, schedules, and policies stay in Dagster regardless of where you run (HPC and non-HPC).
- **Package once, run anywhere** – pixi/pixi-pack builds reproducible environments that deploy to login nodes, edge VMs, or compute partitions.
- **Delegate execution to Slurm** – dagster-slurm translates runs into `sbatch` / `squeue` interactions while respecting quotas and placement rules.
- **See everything in Dagster UI** – queue state, Slurm metrics, and Pipes logs stream back for a single operational console.



EU sovereign GPU cloud does not come out of nowhere

maybe this project can support making HPC systems more accessible

dagster-slurm

Overview **Runs** Assets Automation Deployment

Runs / 50db2417 Success | process_data | Oct 22, 4:14:31 PM | 0:00:07

View asset lineage | View tags and config

Hide not started steps

Re-execute (name:"process_data")

1.4s 2s 3s 4s 5s 6s 7s 8s

process_data

name:"process_data" Hide unselected steps

Preparing (0)

No steps are waiting to execute

Executing (0)

No steps are executing

Errored (0)

Events | stdout | stderr

query:name:"process_data"

Levels (5/6)

TIMESTAMP	OP	EVENT TYPE	INFO
4:14:33.238 PM	process_data	INFO	Starting data processing...
4:14:33.241 PM	process_data	INFO	Input: /path/to/input
4:14:33.244 PM	process_data	INFO	Output: /path/to/output
4:14:33.247 PM	process_data	INFO	Processing complete!
4:14:38.828 PM	process_data	STEP_OUTPUT	Yielded output "result" of type "Any". (Type check passed). rows 1000 processing_time 10s
4:14:38.837 PM	process_data	ASSET_MATERIALI…	Materialized value process_data. asset_key process_data [View Asset] rows 1000 processing_time 10s
4:14:38.858 PM	process_data	ASSET_OBSERVATI…	asset_key process_data [View Asset] deployment_mode local custom_messages_count 0
4:14:38.871 PM	process_data	STEP_SUCCESS	Finished execution of step "process_data" in 6.17s.

Overview Runs Assets Automation Deployment

Assets > process_data

Materialize

Overview Events Checks Lineage Automation

Recent updates

Showing all 2 updates

10/22/2025 10/22/2025

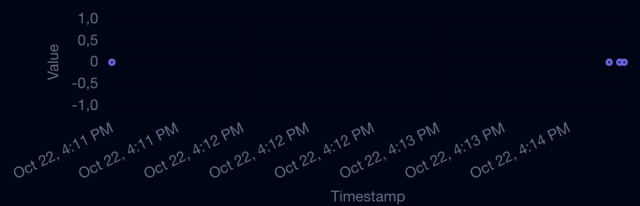
Description

Process data using bash script. Works in all modes (dev/staging/prod) without code changes.

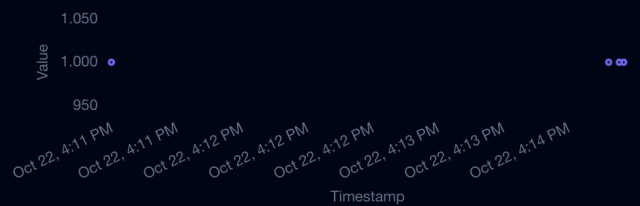
Metadata

Table Plots

custom_messages_count



rows



Lineage

View in graph

Upstream assets

–

Downstream assets

aggregate_results

Definition

Group

shell_example

Code location

dagster_slurm_example

Loaded 5 minutes ago

Source code

Open simple_assets.py in GitHub

Automation details

Sensors

default_automati...condition_sensor

Compute details

Acknowledgements

The EUROCC AI Hackathon https://www.openhackathons.org/s/siteevent/a0CUP000013Tp8f2AC/se000375

We thank the operations teams at the Austrian Scientific Computing (ASC) (VSC-5) and CINECA's Leonardo supercomputer for early feedback, and the Dagster community for discussions around orchestrating HPC workloads. Funding and in-kind support were provided by the Complexity Science Hub Vienna and the Austrian Supply Chain Intelligence Institute. This work was completed in part at the EUROCC AI HACKATHON 2025, part of the Open Hackathons program. The authors would like to acknowledge OpenACC-Standard.org for their support.